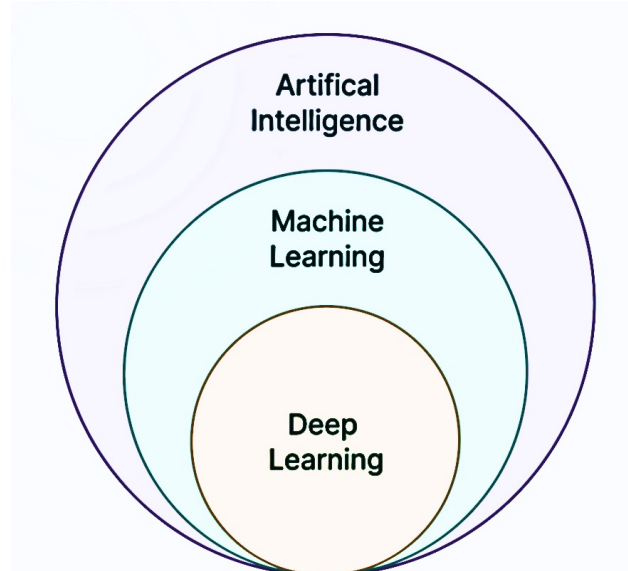


Module 4b

Architecture, Design and Programming of Artificial Neural Network II

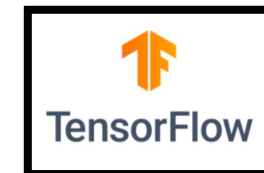


Prof. Emmanuel ADETIBA

**Electrical & Information Engineering Department, College of Engineering,
Covenant University, Ota, Nigeria.**



PROF E. ADETIBA - ARTIFICIAL INTELLIGENCE AND
APPLICATIONS



4.6. Shallow Neural Networks

- ❖ These are neural networks with less depth or number of hidden layers.
- ❖ They usually contain one or two hidden layers and the number of neurons in the hidden layer may vary.
- ❖ Modern deep neural networks can be viewed as more powerful versions of shallow models, with the power achieved through the combination of the basic components into a comprehensive neural architecture.

- ❖ Notably, deep neural architectures are often an overkill in instances where only a small amount of data are available and it is easier to optimize shallow neural models in such settings.
- ❖ Furthermore, the ability to put together the basic units of a shallow model in a clever way is a key skill required to become a deep learning engineer.
- ❖ It is good to learn the properties of the basic models in shallow neural networks, since they are repeatedly used in deep architectures as elementary units of computation.

❖ Notable examples of shallow neural networks are perceptron, Multi-layer Perceptron (MLP), Radial Basis Function (RBF) network, Extreme Learning Machine (ELM), Self Organizing Map (SOM) etc.

❖ Figure 4.18 illustrates the relationship between *accuracy* and *the amount of data* when using *shallow networks* vis-à-vis *deep networks*.

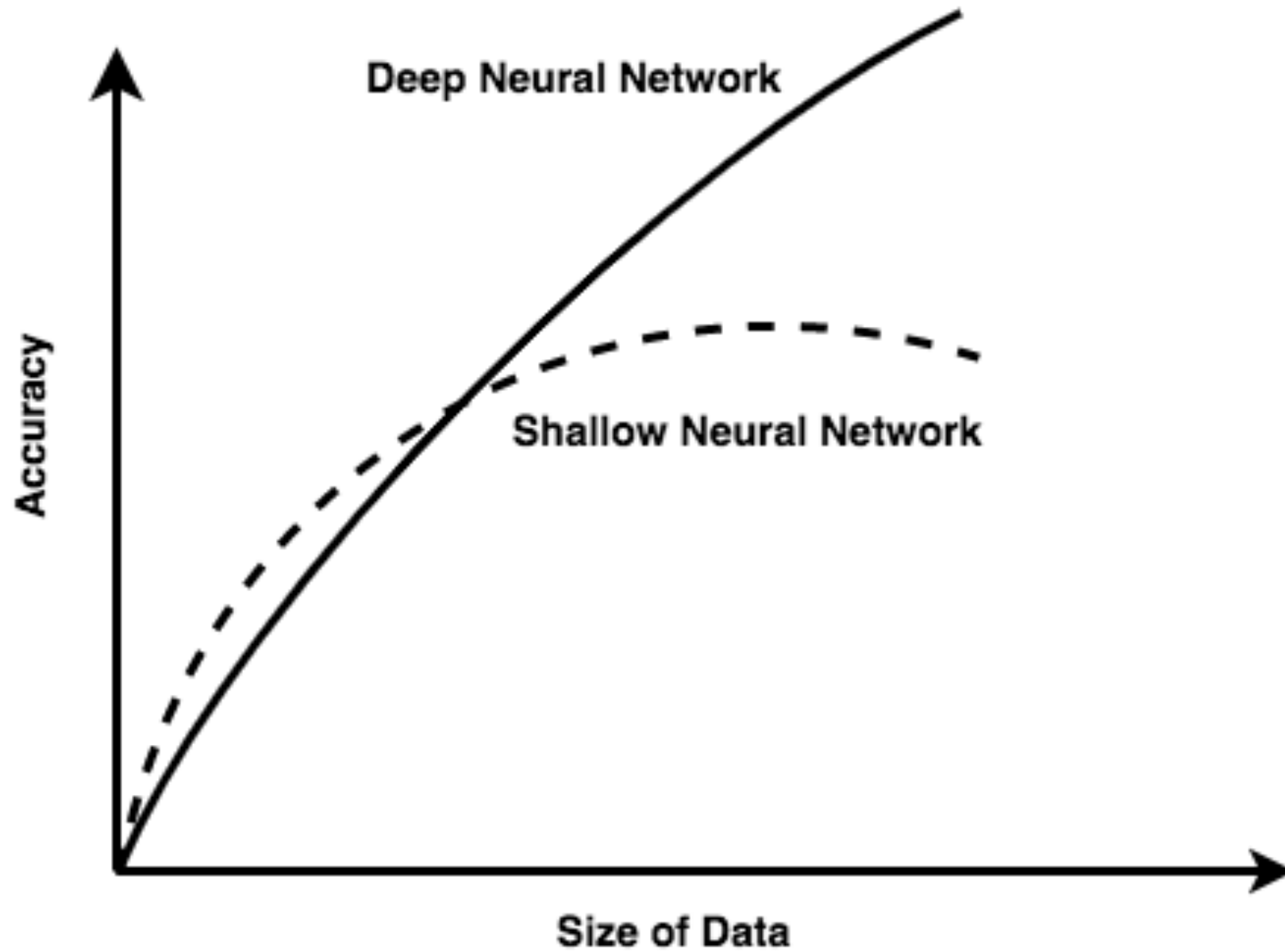


Figure. 4.18: Data Size Versus Accuracy when Using Shallow and Deep Network

4.6.1 Shallow ANN Architecture with Single Layer of Neurons and Matrix Notation

- ❖ Neurons in ANN are normally connected to form a single layer or multiple layers.
- ❖ A sample single layer neural network of S neurons is shown in Fig. 4.19.

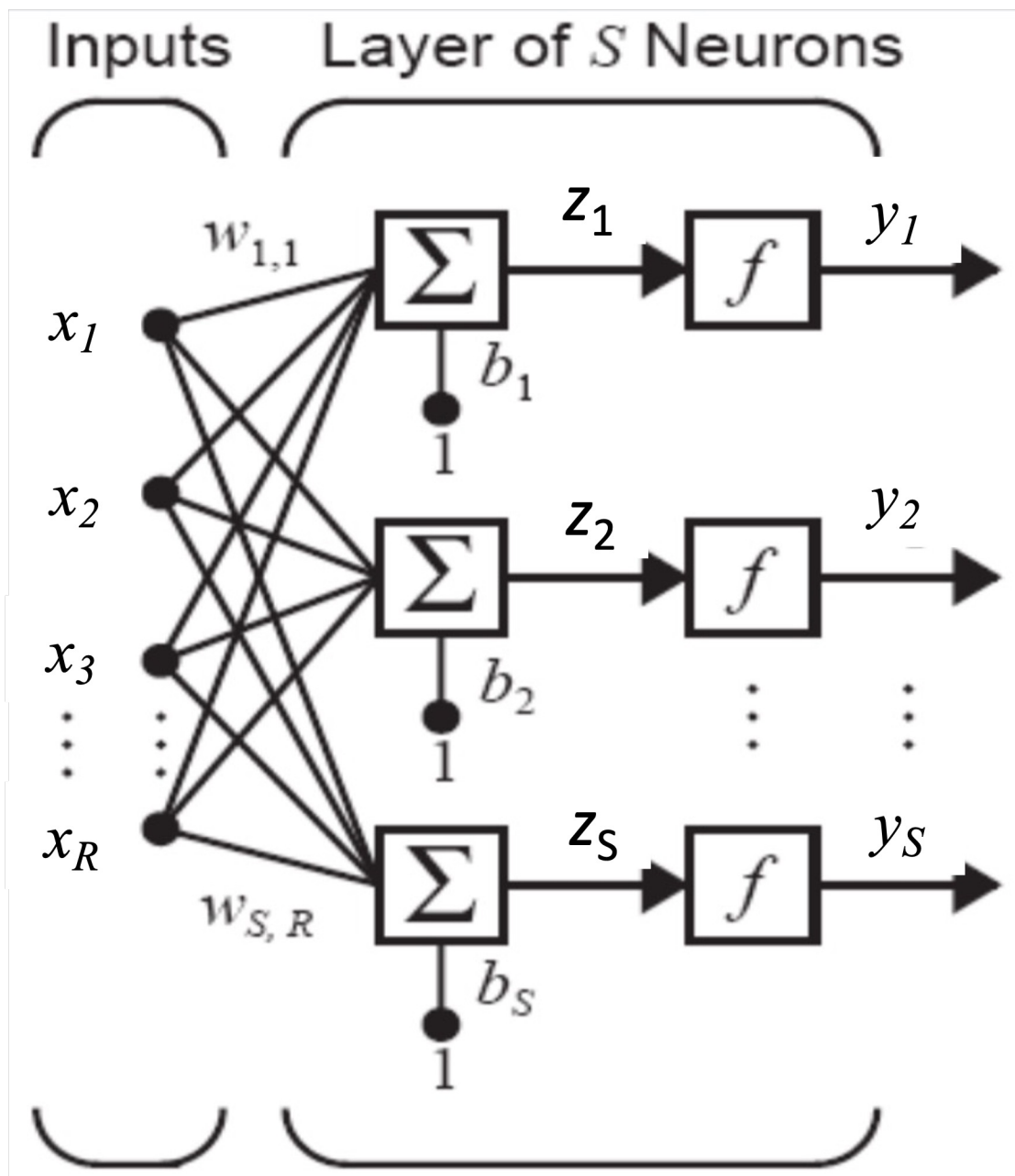


Figure 4.19: A Single Layer of S neurons

❖ As shown in Fig.4.19, each of the R inputs is connected to each of the neurons and the weight matrix $\mathbf{W}$ has S rows.

❖ The input vector $\mathbf{x}$ elements enter the network through the weight matrix $\mathbf{W}$:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ \vdots \\ \vdots \\ x_R \end{bmatrix}, \quad \mathbf{W} = \begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,R} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,R} \\ \vdots & \vdots & & \vdots \\ w_{S,1} & w_{S,2} & \cdots & w_{S,R} \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ \vdots \\ \vdots \\ b_S \end{bmatrix}$$

❖ The output for the single-layer neural network can be represented with matrix notation as:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} \quad (5)$$

$$\mathbf{y} = \mathbf{f}(\mathbf{z}) \quad (6)$$

where,

$$\mathbf{z} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ \vdots \\ \vdots \\ z_s \end{bmatrix},$$

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ \vdots \\ \vdots \\ y_s \end{bmatrix}$$

4.6.2 Shallow ANN Architecture with Multiple-layer of Neurons and Matrix Notation

- ❖ Considering a network with several layers and each layer with its own weight matrix $\mathbf{W}$, bias vector $\mathbf{b}$, a net input vector $\mathbf{z}$ and an output vector $\mathbf{y}$.
- ❖ To distinguish between the different layers, the number of layer can be appended as a superscript to the names for each of the variables as follows:
- ❖ Fig. 4.20 shows a 3-layer neural network.

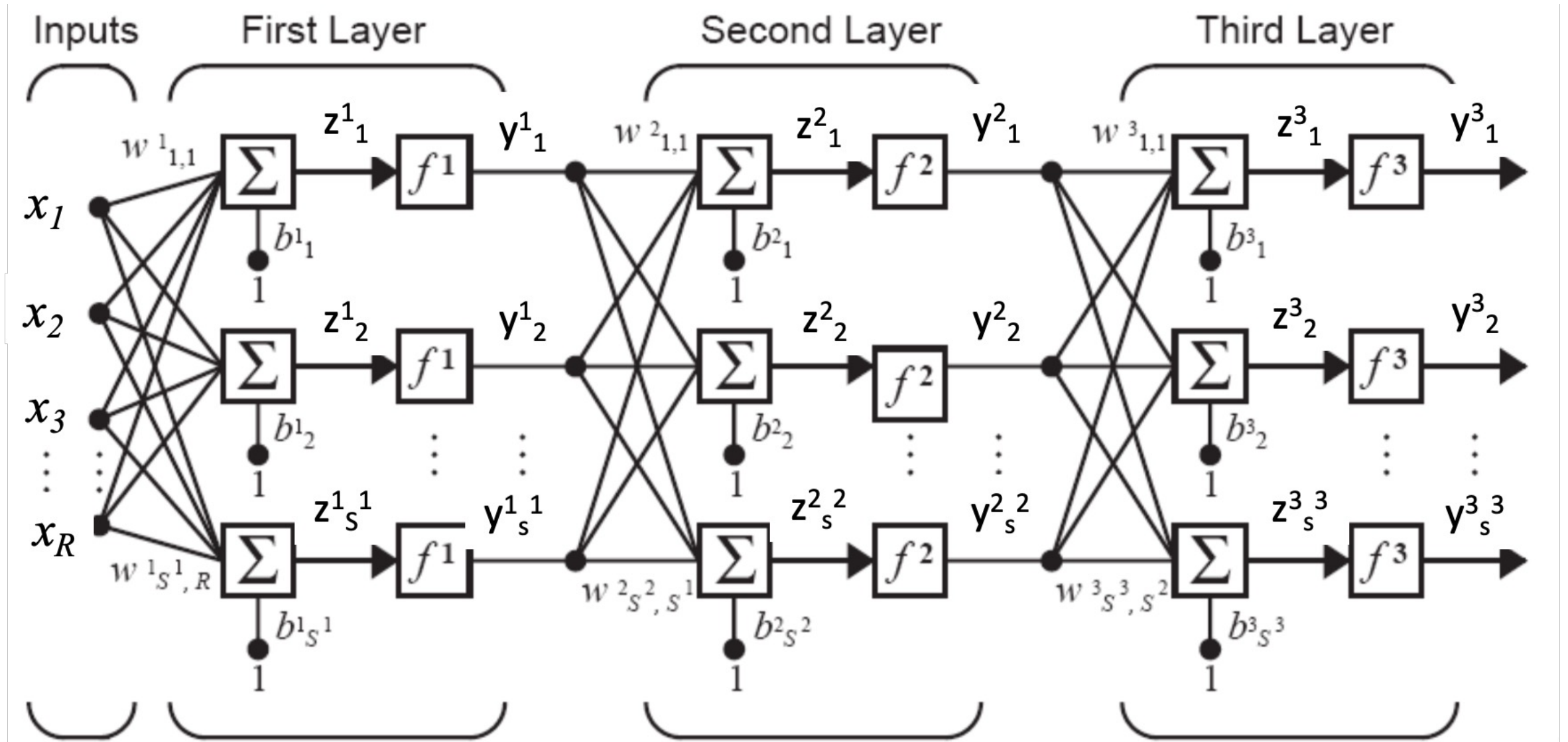


Fig. 4.20: A 3-layer Artificial Neural Network

first layer weight – $\mathbf{W}^1$

second layer weight – $\mathbf{W}^2$

third layer weight – $\mathbf{W}^3$

first layer bias – $\mathbf{b}^1$

second layer bias – $\mathbf{b}^2$

third layer bias – $\mathbf{b}^3$

first layer net input – $\mathbf{z}^1$

second layer net input – $\mathbf{z}^2$

third layer bias net input – $\mathbf{z}^3$

first layer output – $\mathbf{y}^1$

second layer output – $\mathbf{y}^2$

third layer output – $\mathbf{y}^3$

❖ Note that different layers can have different numbers of neurons.

❖ **Output layer** is a layer whose output is the network output.

❖ **Hidden layers** are other layers between the input neurons and the output layer.

❖ First layer output:

$$\mathbf{y}^1 = \mathbf{f}^1(\mathbf{W}^1 \mathbf{x} + \mathbf{b}^1) \quad (7)$$

❖ Second layer output:

$$\mathbf{y}^2 = \mathbf{f}^2(\mathbf{W}^2 \mathbf{y}^1 + \mathbf{b}^2) \quad (8)$$

❖ Third layer output which is the network output is:

$$\mathbf{y}^3 = \mathbf{f}^3(\mathbf{W}^3 \mathbf{y}^2 + \mathbf{b}^3) \quad (9)$$

$$\mathbf{y}^3 = \mathbf{f}^3(\mathbf{W}^3 \mathbf{f}^2(\mathbf{W}^2 \mathbf{f}^1(\mathbf{W}^1 \mathbf{x} + \mathbf{b}^1) + \mathbf{b}^2) + \mathbf{b}^3) \quad (10)$$

4.6.3 Multi-Layer Perceptron

- ❖ Multi-layer Perceptron(MLP) is a supervised feedforward ANN architecture that maps a set of input dataset $X = x_1, x_2, \dots, x_n$ onto an output or a target y . The target could be a scalar or vector ($Y = y_1, y_2, \dots, y_m$) depending on the tasks, which could be classification or regression.
- ❖ MLP consists of three distinct layers: i) input layer ii) hidden layer and iii) output layer as shown in Fig. 4.21.
- ❖ Each of the layers is fully connected to the following one.
- ❖ The neurons in the layers uses non-linear activation functions, except for the input layer neurons.
- ❖ To compute the state of each neuron in MLP, data flows in the forward direction (**i.e. forward pass**) from input to the output layer(see Fig. 4.21).

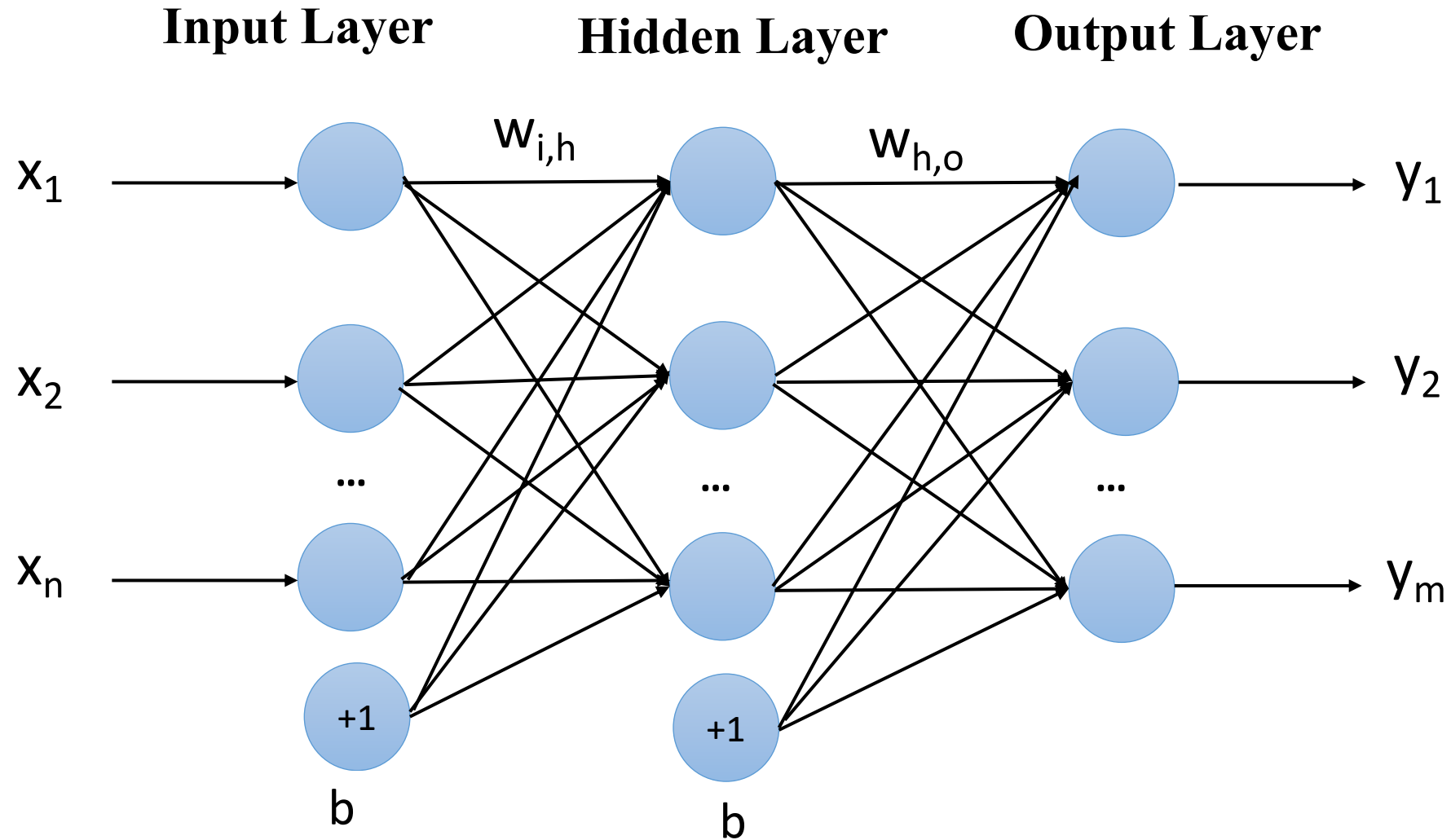


Fig. 4.21: Multilayer Perceptron Architecture with Single Hidden Layer

- i) **Input layer:** This layer, which is the leftmost layer receives the input signal to be processed. It consists of a set of neurons $\{x_1, x_2, \dots, x_n\}$ representing the input features.
- ii) **Hidden layer:** One or more hidden layers are placed in between the input and output layers. The hidden layer forms the true computational engine of the MLP. Each neuron in this layer transforms the values from the previous layer with a weighted linear summation:

$$z_j = \sum_{i=1}^n w_{j,i} x_i + b_i \quad (11)$$

followed by a non-linear activation function:

$$f(z_j): R^n \rightarrow R^m \quad (12)$$

iii) **Output layer:** The output layer receives the values from the last hidden layer and transforms them into output values $(y_1, y_2, \dots, y_m)$.

❖ In MLP, the values of the connecting weights(and biases) are updated using the **Backpropagation** training algorithm.

❖ At the training initialisation phase for MLP, random weights are assigned to all the connections.

❖ These random weights propagate values through the network to produce the actual output in the **forward pass**.

❖ **Backpropagation (BP) algorithm** is a classical algorithm developed by Rumelhart *et. al.* in 1986.

❖ It is a gradient descent based search procedure that works in the **backward pass** during MLP training by measuring the output, calculating the gradient of the error, and adjusting the weights (and biases) in the descending gradient direction in order to reduce the error between the actual (y) and the expected output ($\hat{y}$).

❖ Variants of **gradient descent** algorithm such as *SGD*, *SGD with momentum*, *Adam* and etc are normally used to obtain optimal parameters when training MLP with **BP** algorithm.

4.6.4 Scikit-learn Framework for Implementing Multilayer Perceptron

4.6.4.1 Brief Overview of Scikit-learn

- ❖ Scikit-learn(or sklearn), is a popular Python open-source ML framework with a wide range of ML algorithms and tools for *data preprocessing, model development and model evaluation*.
- ❖ Some of the ML algorithms that are available in Scikit-learn are *perceptron, Multilayer Perceptron (MLP), linear and logistic regression, decision trees, support vector machines, random forests, gradient boosting, k-means clustering etc.*

Example 4.3: A Python code to generate the datasets in Scikit-learn.

```
from sklearn import datasets as ds
# Obtain the list of available datasets
datasetList = ds.__all__
# Print the dataset names
for datasetName in datasetList:
    print(datasetName)
```

❖ ***Some of the outputs of Example 4.3 code are as follows:***

clear_data_home

dump_svmlight_file

fetch_20newsgroups

fetch_20newsgroups_vectorized

fetch_lfw_pairs

fetch_lfw_people

fetch_olivetti_faces

fetch_species_distributions

fetch_california_housing

fetch_covtype

fetch_rcv1

fetch_kddcup99

fetch_openml

get_data_home

load_diabetes

load_digits

load_files

load_iris

load_breast_cancer

load_linnerud

load_sample_image

load_sample_images

load_svmlight_file

load_svmlight_files

load_wine

...etc

4.6.4.2 Some Scikit-learn Hyperparameters for Multi-layer Perceptron

❖ There are several hyperparameters that could be specified when using Sklearn to develop a MLP neural network either for classification or regression tasks. Some of these hyperparameters are described in Table 4.1.

Table 4.1: Some Key Sklearn Hyperparameters for MLP

S/N	Hyperparameters	Description	Example
1	hidden_layer_sizes	This hyperparameter defines the MLP structure. It accepts a list where each element represents the number of neurons in each of the hidden layers.	hidden_layer_sizes=(10, 5) creates a network with two hidden layers with 10 and 5 neurons, respectively.

S/N	Hyperparameters	Description	Example
2	activation	This specifies the activation function for the hidden layers. Available Sklearn activation functions in Sklear are: identity - no activation; logistic - sigmoid function; tanh - hyperbolic tangent function; and relu - rectified linear unit function.	activation='relu'
3	solver	This hyperparameter determines the optimization algorithm used to train the neural network. The solvers in Sklearn are: lbfgs - limited-memory quasi-Newton code for bound-constrained optimization; sgd - stochastic gradient descent, and adam - adaptive moment estimation.	solver='adam'

S/N	Hyperparameters	Description	Example
4	max_iter	This specifies the maximum number of iterations or epochs for training the MLP.	max_iter=1000
5	random_state	This hyperparameter is for setting the random seed for reproducible results.	random_state=42

Module 4b Assignment

Develop a classification model using the breast_cancer dataset in Sklearn and the MLP Classifier.

NOTE:

Your notebook MUST be submitted on MOODLE on or before the next class.
Your notebook should be named as *fullname_matricnumber_mod4b.ipynb*